TEAM BYTEME

AI Meeting Summarizer & Action-Item Tracker

A GenAI productivity tool that turns raw meeting transcripts into structured summaries, tracked action items, and searchable meeting memory.

PYTHON · FASTAPI · STREAMLIT · LLM + VECTOR SEARCH

The Problem

Seen through the eyes of the people who sit in these meetings every week

TEAM LEAD

“What did we actually agree on in that call?”

INDIVIDUAL CONTRIBUTOR

“Who owns this, and by when?”

NEW TEAM MEMBER

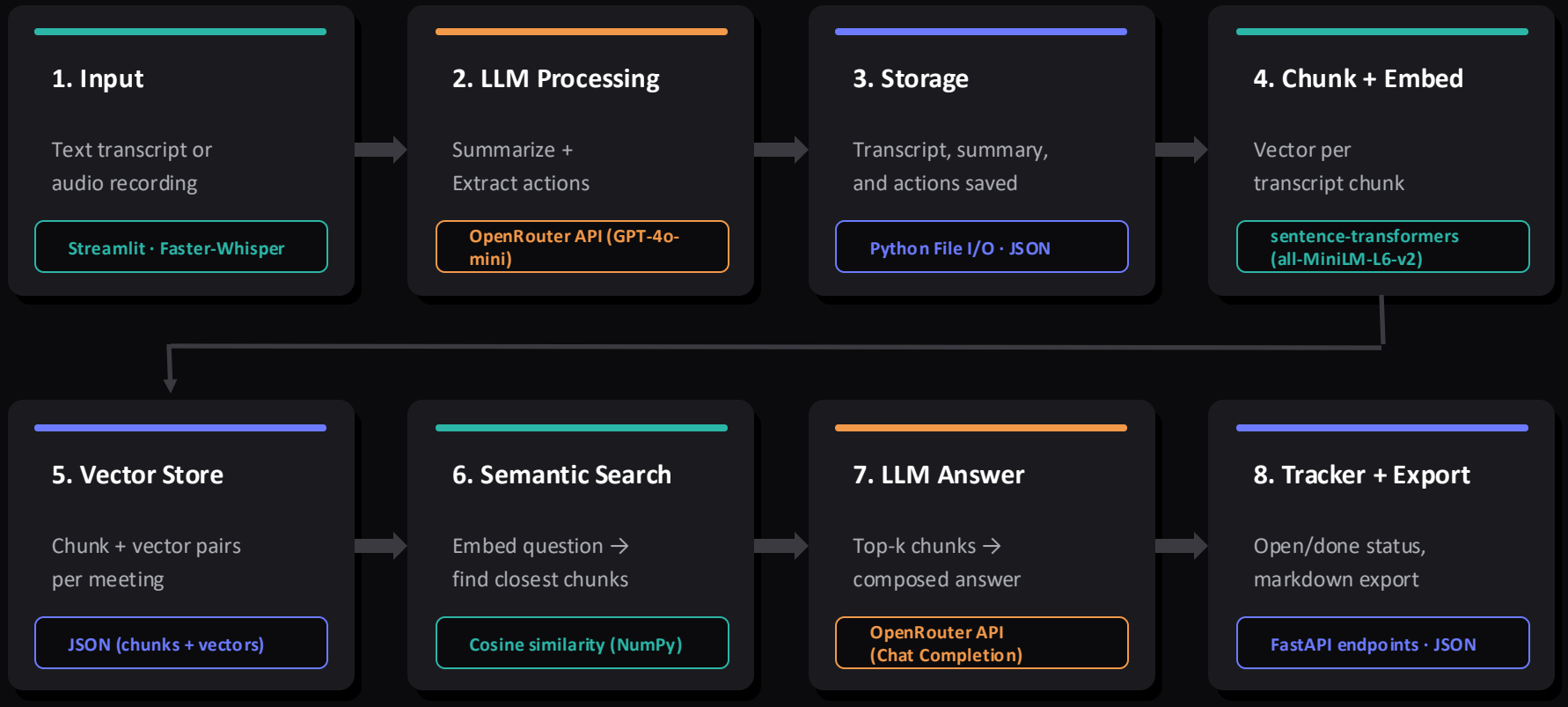
“We keep re-explaining decisions from meetings I wasn’t in.”

The cost to the team

- Decisions get lost because notes are inconsistent or never taken
- Action items fall through the cracks with no clear owner or deadline
- Past discussions aren’t searchable, so the same ground gets re-covered

Data Flow & Architecture

From raw transcript to structured, searchable meeting memory



Steps 1–4 run once per meeting on ingest. Steps 5–8 run on demand — every time a user asks a follow-up question or opens the tracker.

How It Works

Walking through the architecture, stage by stage

01

Ingest

User pastes a transcript (or records/uploads audio, transcribed locally with Whisper) via the Streamlit UI.

02

Summarize

An LLM call (OpenRouter) returns a structured summary: Overview, Decisions, and Risks / Open Questions.

03

Extract Actions

A second LLM call pulls out action items as structured JSON — task, owner, deadline — without inventing missing fields.

04

Store & Embed

Transcript, summary, and actions are saved as JSON. The transcript is chunked and embedded for semantic search.

05

Ask Follow-ups

A question is embedded, matched against stored chunks by cosine similarity, and answered by the LLM using only retrieved context.

06

Track & Export

Action items live in a tracker with open/done status and can be exported as a clean Markdown summary.

Design Trade-offs

Why we chose what we chose — and what we gave up

Area	We Chose	Over	Why
Backend framework	**FastAPI**	*Flask*	Async support, automatic Swagger docs, and built-in request validation via Pydantic — less boilerplate for a JSON API.
Embeddings	**sentence-transformers (local)**	*OpenAI / Cohere embedding APIs*	Free, runs offline, no extra API key — traded a small quality ceiling for zero cost and simplicity.
Extraction logic	**LLM-based extraction**	*Regex / keyword heuristics*	Far more reliable at handling natural phrasing and leaving owner/deadline blank when genuinely unstated.
Storage	**Flat JSON files**	*SQL / vector database*	Fast to build and fully sufficient at this scale; a real DB (e.g. FAISS, Postgres) is the natural next step at production scale.

Thank You

AI Meeting Summarizer & Action-Item Tracker

Team ByteMe